

GUIDE DE MAINTENANCE

DONCIEL Trading Journal

Guide technique de maintenance pour l'application DONCIEL Trading Journal. Ce document couvre l'architecture, les procédures de mise à jour, la sauvegarde et le dépannage courant.

Version 1.0

Date Mars 2026

Auteur Equipe DONCIEL

Table des matieres

1. Introduction	3
2. Elements critiques a conserver	3
2.1 Base de donnees PostgreSQL (Neon)	3
2.2 Token GitHub	4
2.3 Compte administrateur	4
3. Architecture de l'application	4
3.1 Structure des dossiers clés	4
4. Procedures de mise a jour	5
4.1 Modifier le code localement	5
4.2 Pousser sur GitHub	5
4.3 Deploiement Vercel	5
4.4 Modifier le schema Prisma	6
5. Sauvegarde et restauration	6
5.1 Sauvegarde de la base de donnees	7
5.2 Sauvegarde du code source	7
5.3 Sauvegarde du fichier .env	7
6. Depannage courant	7
6.1 Verification de la connexion base de donnees	8
6.2 Reinitialisation de Prisma	8

1. Introduction

DONCIEL Trading Journal est une application web de journal de trading permettant aux traders de suivre, analyser et optimiser leurs operations sur les marches financiers. L'application offre un suivi detaille des trades, des statistiques de performance et une interface intuitive basee sur les technologies web modernes.

Ce guide de maintenance a pour objectif de fournir toutes les informations necessaires pour assurer le bon fonctionnement, la mise a jour et le depannage de l'application. Il s'adresse aux administrateurs et developpeurs charges de la maintenance technique.

Objectif principal : Garantir la continuite de service, la securite des donnees et la facilite de maintenance de l'application DONCIEL Trading Journal.

2. Elements critiques a conserver

Les elements suivants sont essentiels au fonctionnement de l'application. Toute perte ou compromission de ces elements peut entrainer une indisponibilite complete du service.

Element	Description	Details
Base de donnees PostgreSQL (Neon)	Stockage principal de toutes les donnees applicatives	URL de connexion dans DATABASE_URL
Token GitHub	Acces au depot de code source	Droits de lecture/ecriture sur le depot
Variables d'environnement (.env)	Configuration sensible de l'application	DATABASE_URL et autres secrets
Compte administrateur	Acces de gestion a l'application	doncielkabwe@gmail.com
Repertoire GitHub	Depot de code source principal	https://github.com/Prince-ilunga/donciel-tm

Tableau 1 : Elements critiques de l'application

2.1 Base de donnees PostgreSQL (Neon)

La base de donnees hebergee sur Neon constitue le coeur de l'application. Elle stocke l'ensemble des trades, les statistiques, les profils utilisateurs et les parametres de configuration. La chaine de connexion (DATABASE_URL) est l'element le plus sensible du systeme.

Attention : La perte de la chaine DATABASE_URL entraine l'impossibilite totale de se connecter a la base de donnees. Conservez cette information dans un lieu sur et redonde.

2.2 Token GitHub

Le token GitHub est necessaire pour pousser le code source vers le depot. Sans ce token, il est impossible de deployer de nouvelles mises a jour. Assurez-vous que le token dispose des permissions adequates (repo, workflow).

2.3 Compte administrateur

Le compte administrateur (doncielkabwe@gmail.com) dispose d'un acces privilegie a l'ensemble des fonctionnalites de gestion. La perte de l'accès a cette adresse email compromet la capacite a administrer l'application.

3. Architecture de l'application

L'application DONCIEL Trading Journal est construite sur une architecture moderne basee sur le framework Next.js avec un rendu cote serveur, couple a une base de donnees PostgreSQL geree via Prisma ORM.

Composant	Technologie	Role
Framework	Next.js 16 + App Router	Application web avec SSR/SSG
ORM	Prisma ORM	Mapping objet-relationnel pour PostgreSQL
Base de donnees	PostgreSQL (Neon)	Stockage persistant des donnees
UI Components	shadcn/ui + Tailwind CSS	Composants d'interface reutilisables
Authentification	JWT cookies	Gestion des sessions securisees

Tableau 2 : Stack technologique

3.1 Structure des dossiers clés

La structure du projet suit les conventions Next.js avec une organisation modulaire des composants et des routes API :

Dossier	Contenu
src/app/api/	Routes API (endpoints REST)
src/components/	Composants React réutilisables
prisma/	Schema Prisma et migrations
src/app/	Pages et layouts (App Router)
src/lib/	Utilitaires et fonctions partagées
public/	Fichiers statiques (images, icônes)

Tableau 3 : Structure des dossiers

4. Procédures de mise à jour

Les mises à jour de l'application suivent un processus structuré en plusieurs étapes. Le déploiement est automatisé via Vercel à chaque push sur la branche principale.

4.1 Modifier le code localement

Pour apporter des modifications au code source :

- Cloner le dépôt : `git clone https://github.com/Prince-ilunga/donciel-tm`
- Installer les dépendances : `npm install`
- Créer une branche de fonctionnalité : `git checkout -b feature/nom-fonctionnalite`
- Apporter les modifications nécessaires
- Tester localement : `npm run dev`

4.2 Pousser sur GitHub

Une fois les modifications validées localement, poussez le code vers le dépôt distant :

```
git add . && git commit -m "message descriptif" && git push origin main
```

Important : Le message de commit doit être descriptif et refléter précisément la nature des modifications apportées.

4.3 Déploiement Vercel

Le déploiement sur Vercel est automatique. À chaque push sur la branche main, Vercel détecte les changements et lance un nouveau build en production. Aucune intervention manuelle n'est requise pour

le deploiement routine.

- Vercel detecte automatiquement les push sur main
- Le build est lance automatiquement
- En cas d'erreur, consulter les logs sur le tableau de bord Vercel

4.4 Modifier le schema Prisma

Les modifications du schema de base de donnees suivent un processus specifique pour eviter toute perte de donnees :

Eta pe	Commande	Description
1	Editer prisma/schema.prisma	Modifier le schema selon les besoins
2	npx prisma db push	Appliquer les changements a la base Neon
3	npx prisma generate	Regenerer le client Prisma

Tableau 4 : Procedure de modification du schema Prisma

Attention : La commande `prisma db push` applique les changements directement sans migration reversible. Pour les environnements de production, preferer `prisma migrate deploy`.

5. Sauvegarde et restauration

Une strategie de sauvegarde rigoureuse est essentielle pour garantir la resilience de l'application face aux incidents. Trois elements doivent etre sauvegardes regulierement.

Element	Methode de sauvegarde	Frequence recommandee
Base de donnees Neon	pg_dump vers fichier SQL	Quotidienne
Code source	GitHub comme sauvegarde	A chaque commit
Fichier .env	Copie securisee hors ligne	A chaque modification

Tableau 5 : Strategie de sauvegarde

5.1 Sauvegarde de la base de donnees

Utilisez la commande `pg_dump` pour creer une sauvegarde complete de la base de donnees Neon :

```
pg_dump "DATABASE_URL" --no-owner --no-privileges > backup_$(date +%Y%m%d).sql
```

Pour restaurer une sauvegarde :

```
psql "DATABASE_URL" < backup_20260301.sql
```

5.2 Sauvegarde du code source

Le depot GitHub sert de sauvegarde principale du code source. Chaque commit cree un point de restauration. Pour une securite supplementaire, il est recommande de cloner le depot sur un support de stockage local.

5.3 Sauvegarde du fichier .env

Le fichier `.env` contient les variables d'environnement critiques, notamment la chaine de connexion `DATABASE_URL`. Ce fichier ne doit jamais etre commite dans le depot Git. Conservez une copie securisee dans un gestionnaire de mots de passe ou un coffre-fort numerique.

Regle absolue : Ne jamais commiter le fichier `.env` dans le depot Git. Ajoutez-le toujours au fichier `.gitignore`.

6. Depannage courant

Cette section presente les problemes les plus frequemment rencontres et leurs solutions.

Probleme	Cause probable	Solution
Erreur de connexion a la base de donnees	DATABASE_URL incorrecte ou expiree	Verifier la valeur de DATABASE_URL dans le fichier .env
Erreur Prisma lors du build	Client Prisma non genere	Executer <code>npx prisma generate</code>
Build Vercel echoue	Erreur de compilation ou variable manquante	Consulter les logs Vercel et verifier les variables d'environnement
Port 3000 deja occupe	Processus existant sur le port 3000	Executer <code>fuser -k 3000/tcp</code>
Page blanche en production	Erreur d'hydratation React	Verifier les logs navigateur et les props du composant
Session expiree	Cookie JWT expire	Se reconnecter via la page de login

Tableau 6 : Guide de depannage

6.1 Verification de la connexion base de donnees

En cas de probleme de connexion a la base de donnees, suivez ces etapes :

- Verifiez que la variable DATABASE_URL est presente dans le fichier .env
- Testez la connexion directement avec `psql "$DATABASE_URL"`
- Verifiez que la base Neon est active (pas en mode veille)
- Consultez le tableau de bord Neon pour verifier le statut

6.2 Reinitialisation de Prisma

Si les erreurs Prisma persistent apres la generation du client :

- `npx prisma generate` - Regenerer le client
- `npx prisma db push` - Synchroniser le schema
- Supprimer `node_modules/.prisma` et regenerer

7. Contact et support

Pour toute question ou assistance technique concernant l'application DONCIEL Trading Journal, veuillez contacter :

Canal	Detail
Email administrateur	doncielkabwe@gmail.com
Depot GitHub	https://github.com/Prince-ilunga/donciel-tm
Hebergement	Vercel (deploiement automatique)
Base de donnees	Neon PostgreSQL (cloud)

Tableau 7 : Informations de contact